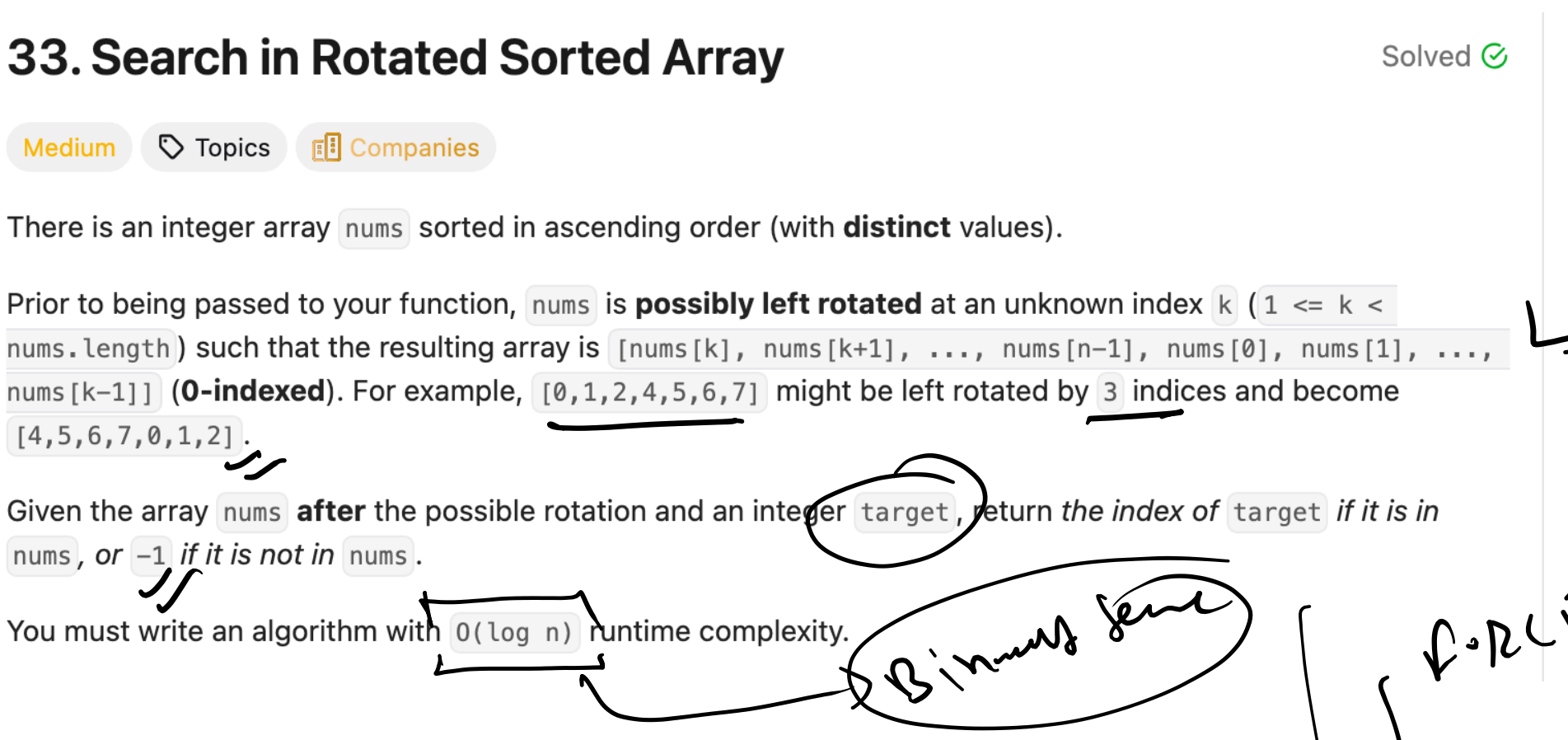
Lec-20
10/11/26 10:14 AM

① next Permutation
(ii) Rotate Sorted Array
(iii) get the max Score
(iv) repair cars → method
Sort color → 0,1,2
(v) maximum char freq
(vi) Help Ramu
(vii) Diagonal Traverse



33. Search in Rotated Sorted Array

Solved ✓

Medium Topics Companies

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **possibly left rotated** at an unknown index `k` (`1 <= k < nums.length`) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be left rotated by `3` indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return *the index of* `target` *if it is in* `nums`, *or* `-1` *if it is not in* `nums`.

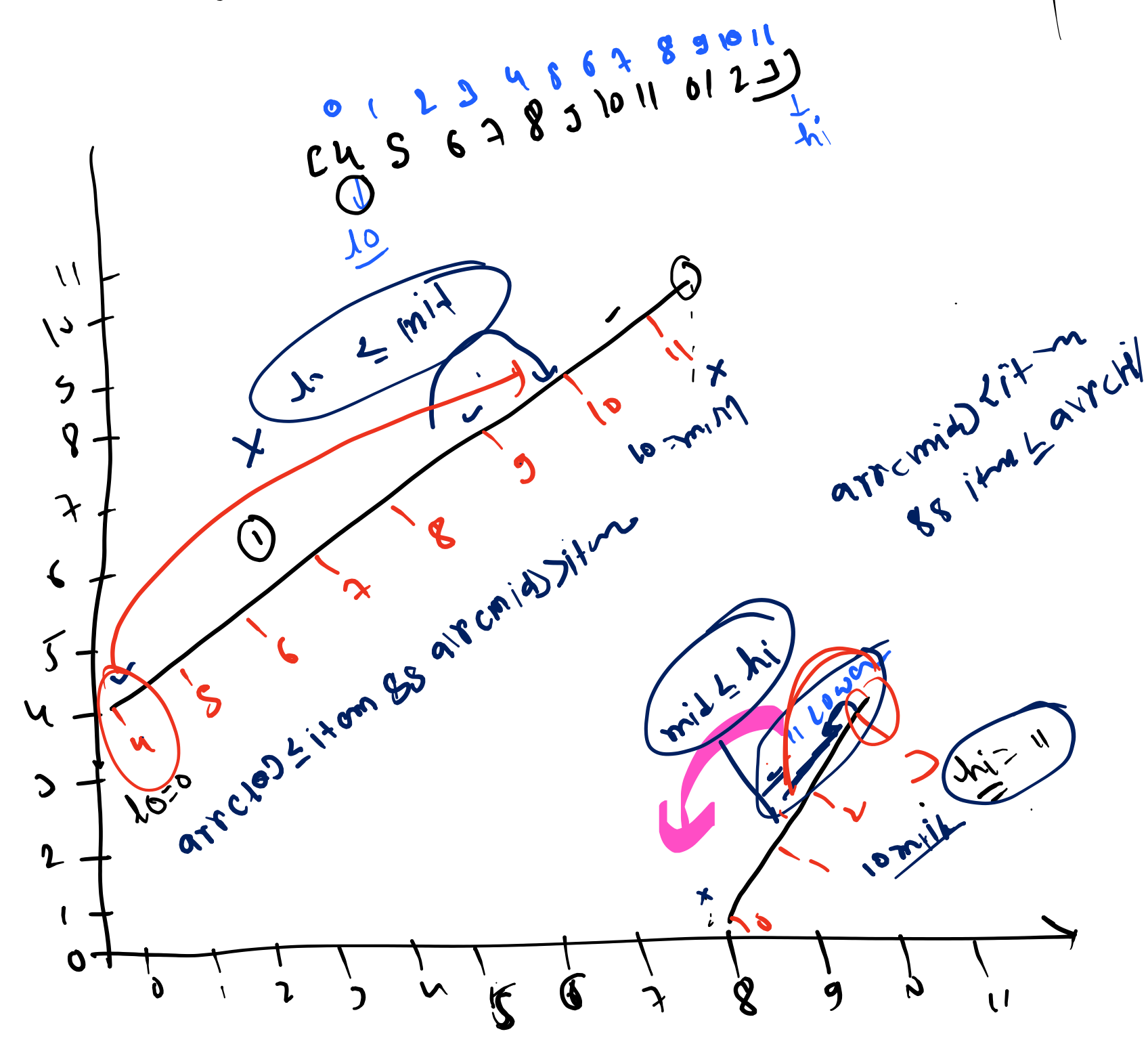
You must write an algorithm with `O(log n)` runtime complexity.

Binary Search

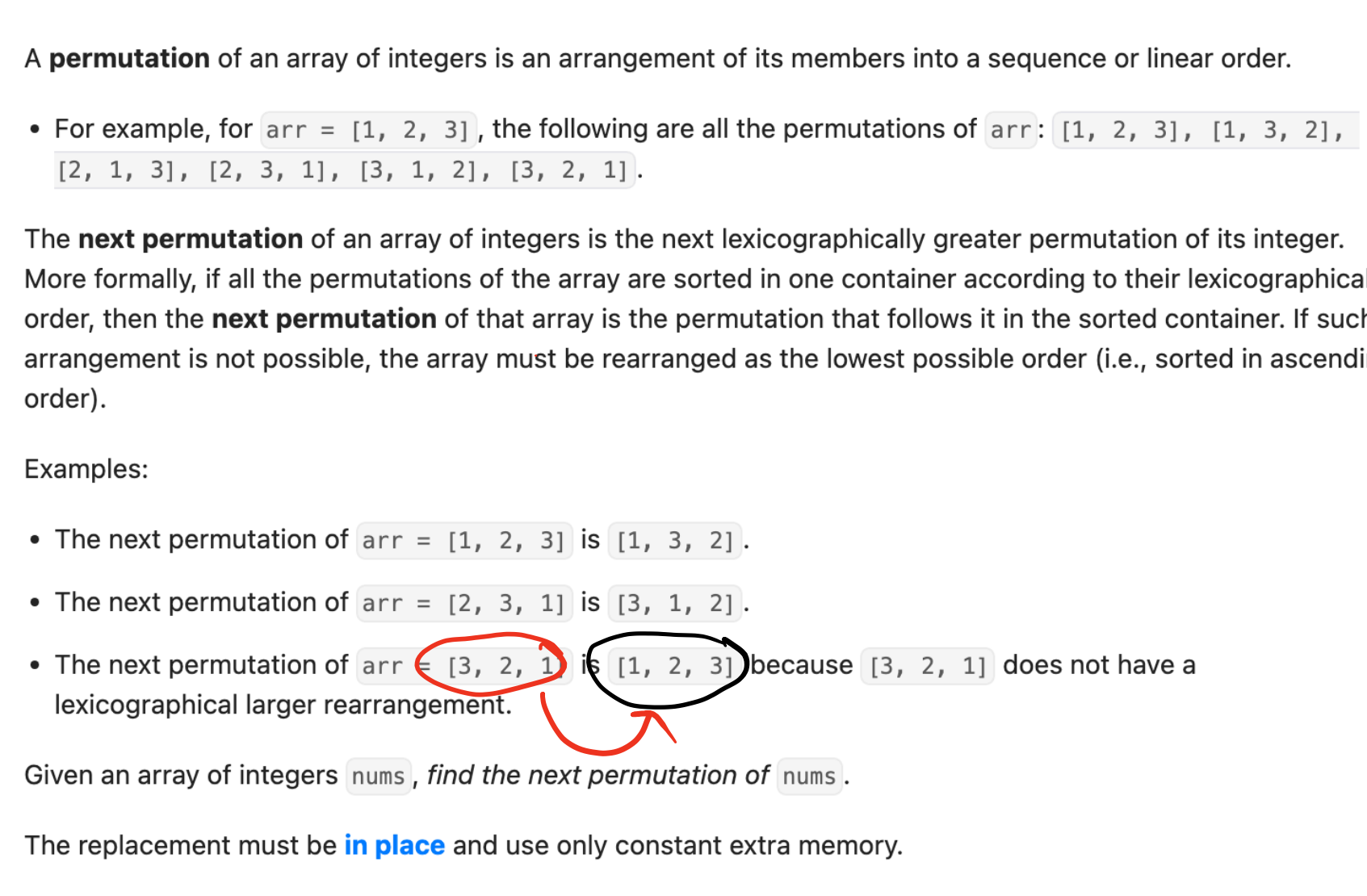
[0 1 2 4 5 6 7]
[4 5 6 7 0 1 2 3]
0 1 2 3 4 5 6

for(i=0 i<n i++){
if(nums[i]==target)
return
}
return -1

[4 5 6 7 0 1 2]
0 1 2 3 4 5 6



while (lo ≤ hi){
mid = (lo + hi) / 2
if (arr[mid] == item){
return mid
}
if (arr[mid] >= arr[lo]){
// part 2
}
else{
}
}



A **permutation** of an array of integers is an arrangement of its members into a sequence or linear order.

- For example, for `arr = [1, 2, 3]`, the following are all the permutations of `arr`: `[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]`.

The **next permutation** of an array of integers is the next lexicographically greater permutation of its integer. More formally, if all the permutations of the array are sorted in one container according to their lexicographical order, then the **next permutation** of that array is the permutation that follows it in the sorted container. If such arrangement is not possible, the array must be rearranged as the lowest possible order (i.e., sorted in ascending order).

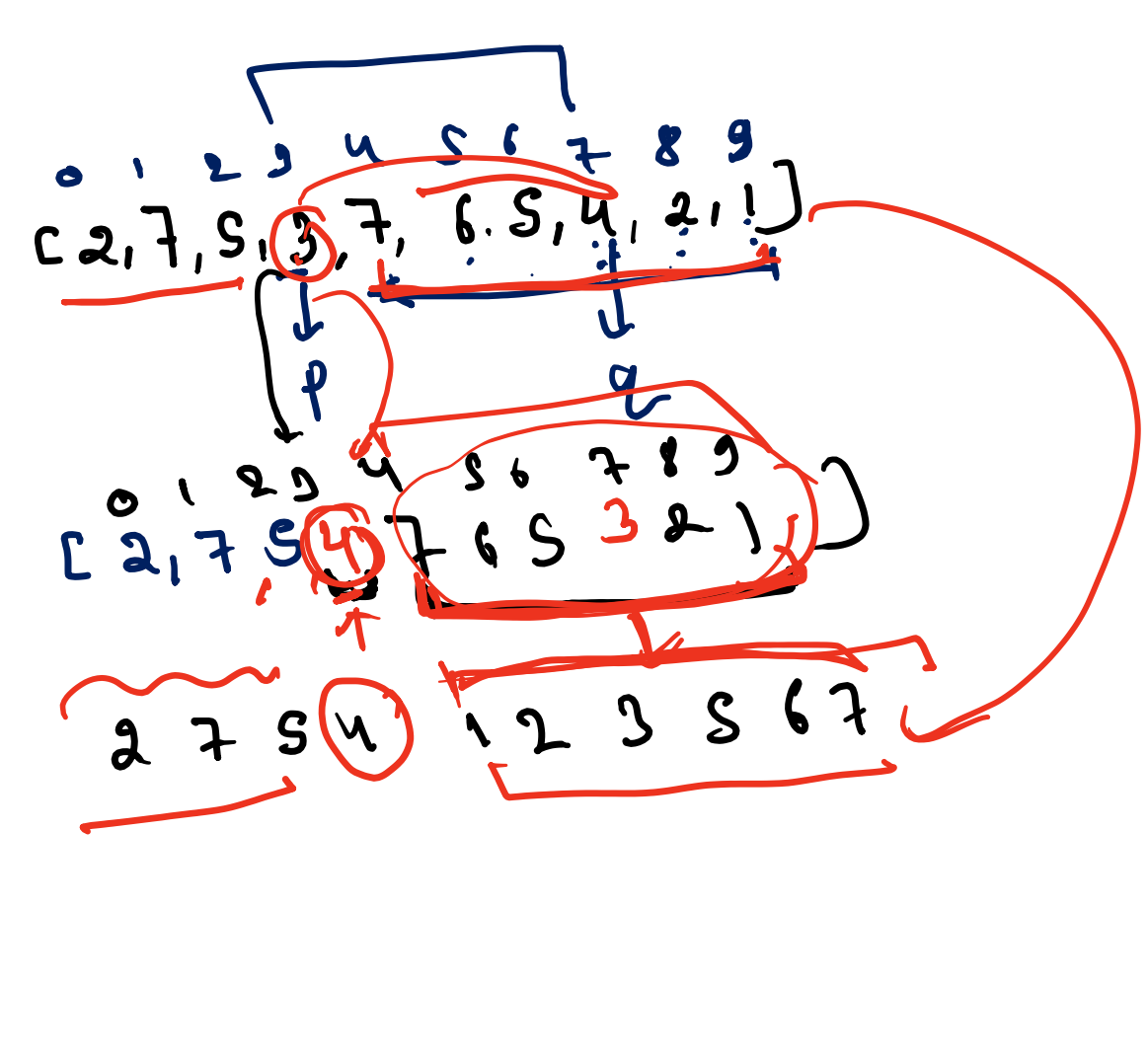
Examples:

- The next permutation of `arr = [1, 2, 3]` is `[1, 3, 2]`.
- The next permutation of `arr = [2, 3, 1]` is `[3, 1, 2]`.
- The next permutation of `arr = [3, 2, 1]` is `[1, 2, 3]` because `[3, 2, 1]` does not have a lexicographical larger rearrangement.

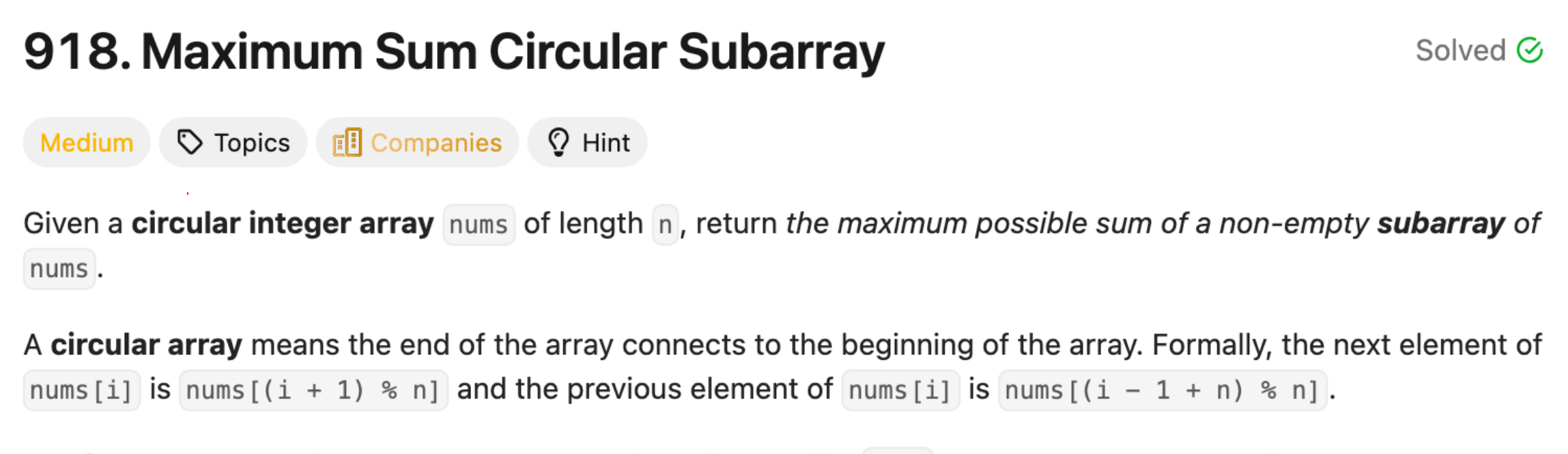
Given an array of integers `nums`, *find the next permutation of* `nums`.

The replacement must be **in place** and use only constant extra memory.

[1, 2, 3]
123 →
[1, 3, 2]
123
132
213
231
312
321
[3, 2, 1] → 321
(1, 2, 3)
[3, 2, 1]



N = 10
→ n-2
p = -1
i = n-2 { if(arr[i] < arr[i+1]){
p = i
break
}
}
for(i = n-1 i > p i--){
if(arr[i] > arr[p]){
q = i
break
}
}
Swap(p, q)
Rev (p+1, n-1)



918. Maximum Sum Circular Subarray

Solved ✓

Medium Topics Companies Hint

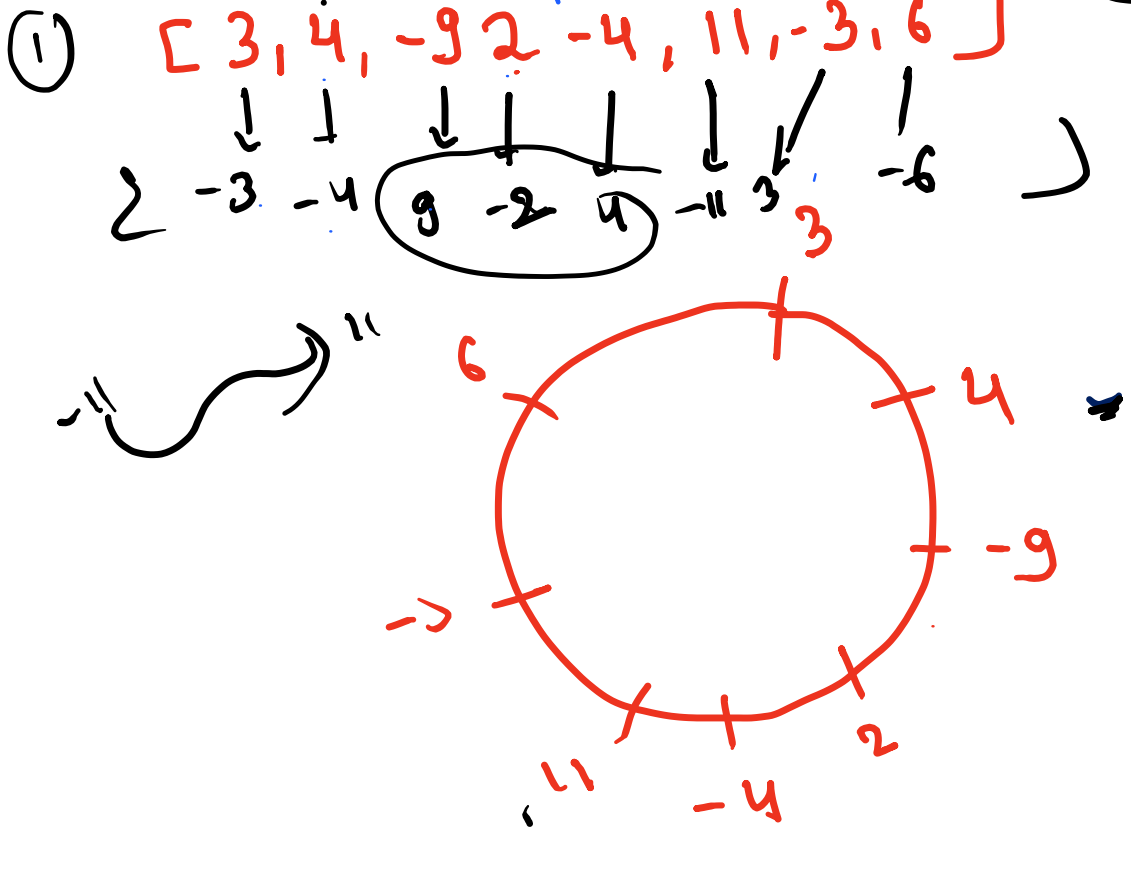
Given a **circular integer array** `nums` of length `n`, return *the maximum possible sum of a non-empty* ***subarray*** *of* `nums`.

A **circular array** means the end of the array connects to the beginning of the array. Formally, the next element of `nums[i]` is `nums[(i + 1) % n]` and the previous element of `nums[i]` is `nums[(i - 1 + n) % n]`.

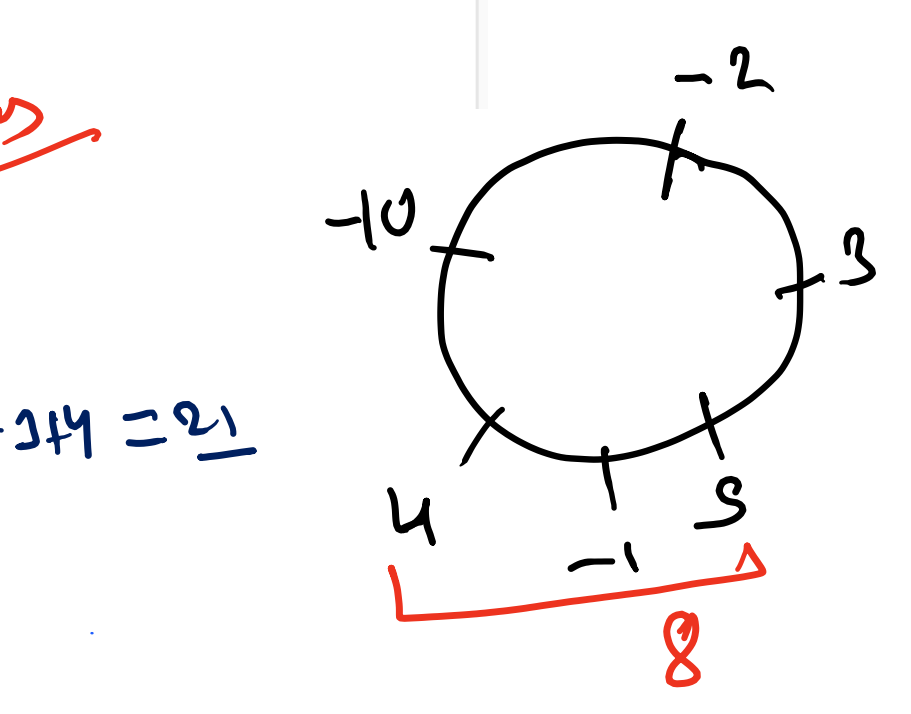
A **subarray** may only include each element of the fixed buffer `nums` at most once. Formally, for a subarray `nums[i], nums[i + 1], ..., nums[j]`, there does not exist `i <= k1`, `k2 <= j` with `k1 % n == k2 % n`.

21 - 3+4 - 9+2-4+11 - 3+6 = (21 - 11) = [10 + 11] = 21

① [3, 4, -9, 2, -4, 11, -3, 6]
{ -3 -4 9 -2 4 -11 3 -6 }
Kadane's



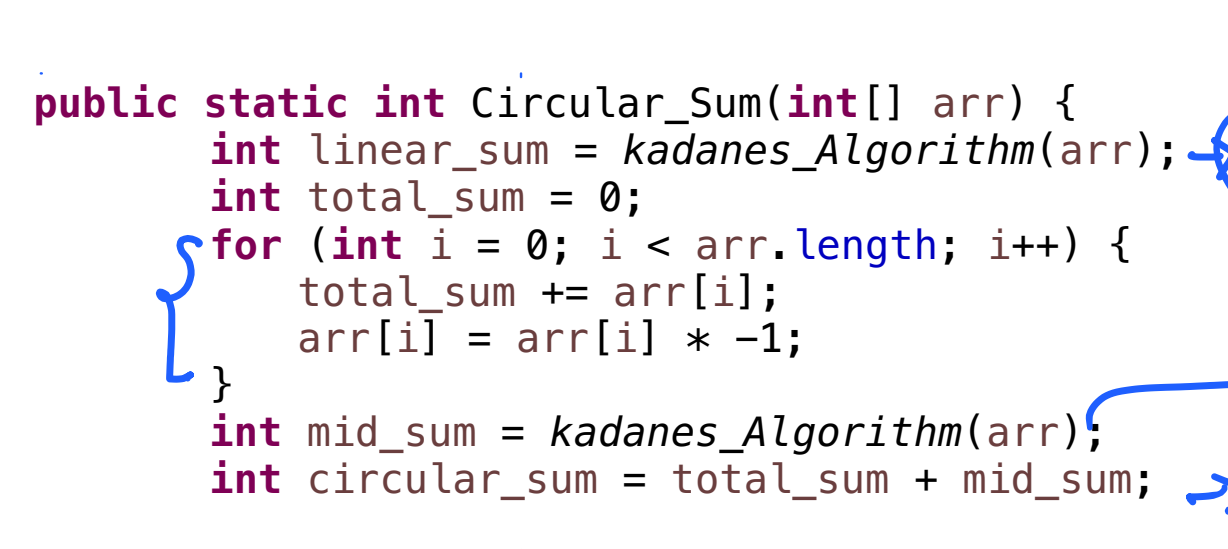
11-3+6+3+4 = 21



(ii) [-2, -3, 5, -1, 4, -10]
→ Linear max
→ Circular max

[3 4 -9 2 -4 11 → 6 3 4 -9 2 -4 11 -3 6]

-2-3-4 = -9

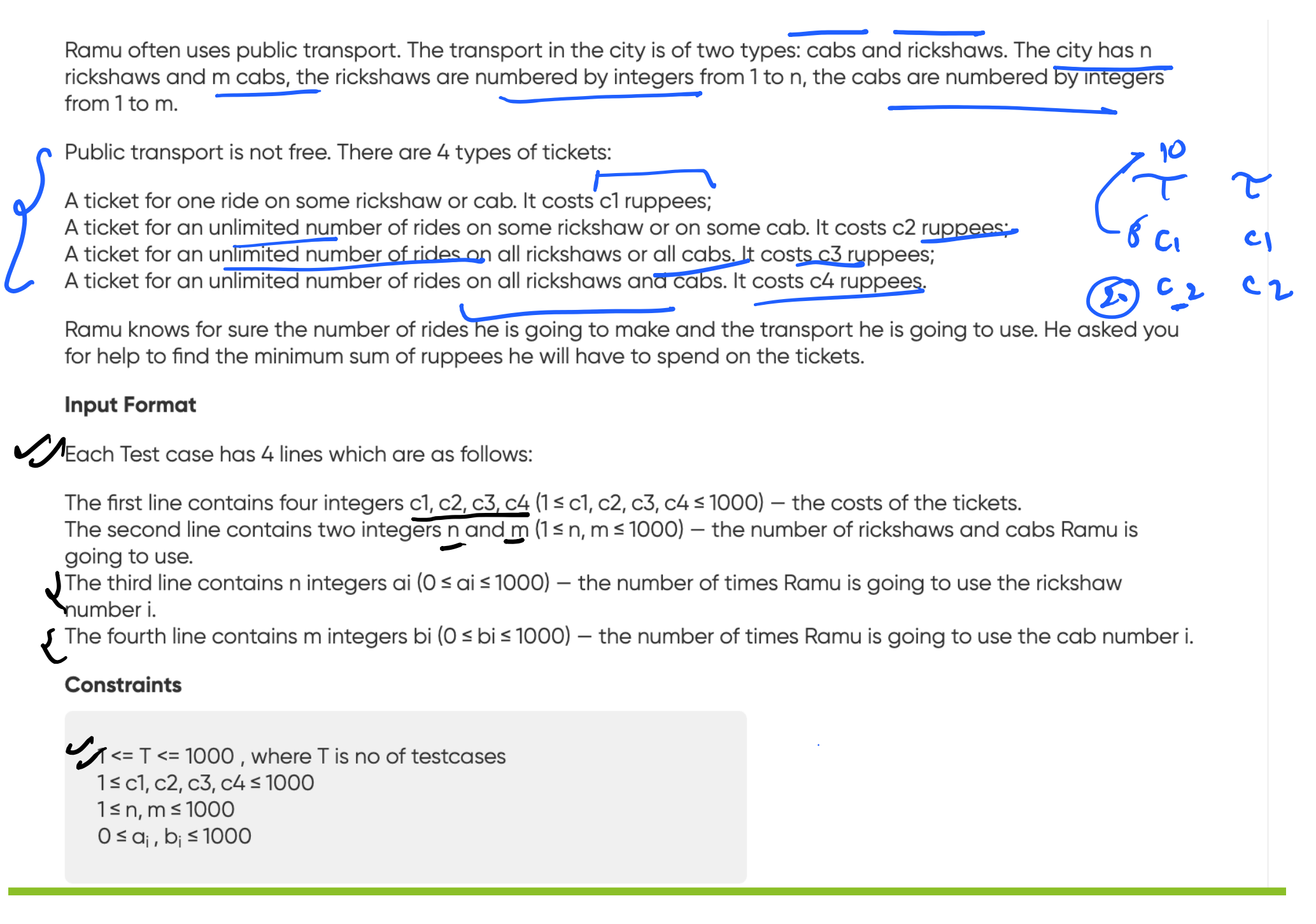


```
public static int Circular_Sum(int[] arr) {
        int linear_sum = kadanes_Algorithm(arr);
        int total_sum = 0;
        for (int i = 0; i < arr.length; i++) {
            total_sum += arr[i];
            arr[i] = arr[i] * -1;
        }
        int mid_sum = kadanes_Algorithm(arr);
        int circular_sum = total_sum + mid_sum;

        return Math.max(linear_sum, circular_sum);

    }
```

-2
[-2, -3, -4]
[2 3 4] → 9
→ -9+9 = 0
(-2 0)



Ramu often uses public transport. The transport in the city is of two types: cabs and rickshaws. The city has n rickshaws and m cabs, the rickshaws are numbered by integers from 1 to n, the cabs are numbered by integers from 1 to m.

Public transport is not free. There are 4 types of tickets:

A ticket for one ride on some rickshaw or cab. It costs c1 ruppees;
A ticket for an unlimited number of rides on some rickshaw or on some cab. It costs c2 ruppees;
A ticket for an unlimited number of rides on all rickshaws or all cabs. It costs c3 ruppees;
A ticket for an unlimited number of rides on all rickshaws and cabs. It costs c4 ruppees.

Ramu knows for sure the number of rides he is going to make and the transport he is going to use. He asked you for help to find the minimum sum of ruppees he will have to spend on the tickets.

Input Format

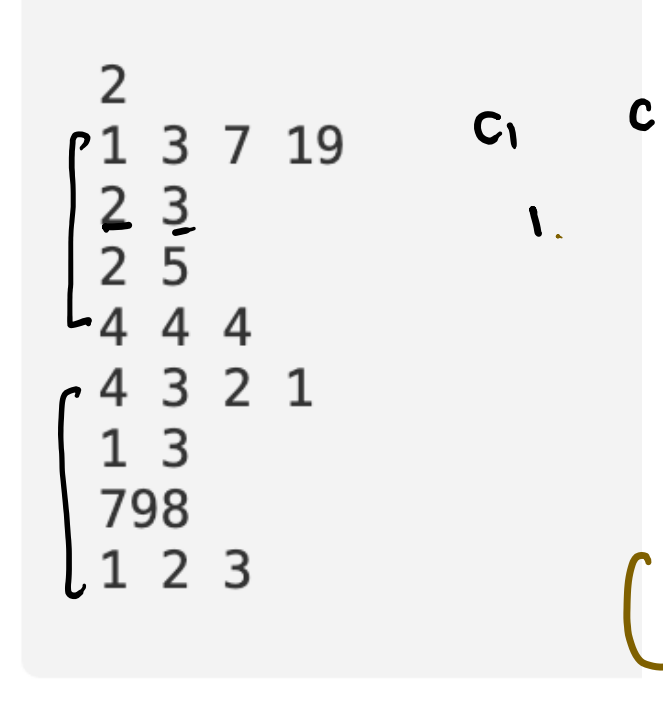
Each Test case has 4 lines which are as follows:

The first line contains four integers c1, c2, c3, c4 (1 ≤ c1, c2, c3, c4 ≤ 1000) — the costs of the tickets.
The second line contains two integers n and m (1 ≤ n, m ≤ 1000) — the number of rickshaws and cabs Ramu is going to use.
The third line contains n integers ai (0 ≤ ai ≤ 1000) — the number of times Ramu is going to use the rickshaw number i.
The fourth line contains m integers bi (0 ≤ bi ≤ 1000) — the number of times Ramu is going to use the cab number i.

Constraints

1 <= T <= 1000 , where T is no of testcases
1 ≤ c1, c2, c3, c4 ≤ 1000
1 ≤ n, m ≤ 1000
0 ≤ a_i , b_i ≤ 1000

10 T T
8 c1 c1
Σ c2 c2



```
2
1 3 7 19
2 3
2 5
4 4 4
4 3 2 1
1 3
798
1 2 3
```

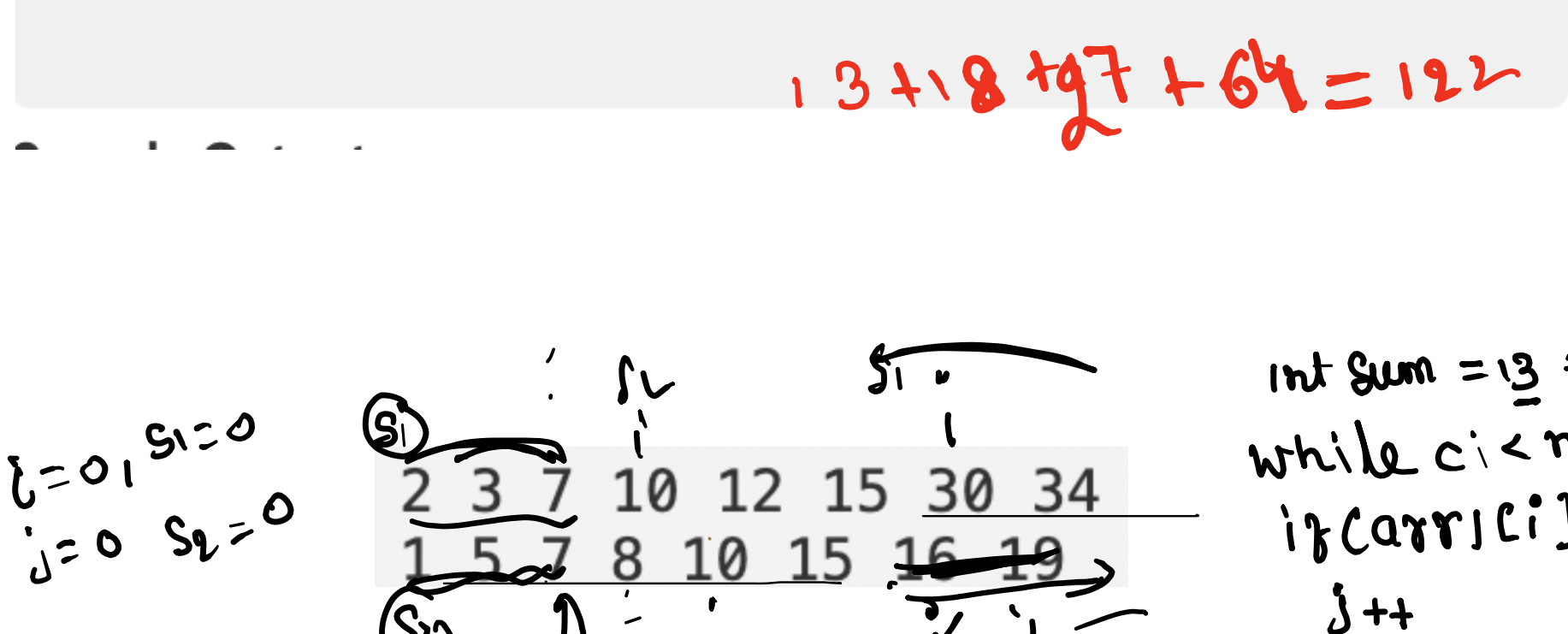
c1 c2 c3 c4
1 3 7 19

Rn = 2 Cm = 3
5 [2, 5] [4, 4, 4]

(2×1, 3) = 2
(5×1, 3) = 3
5
min(5, 7) → 5

(4×1, 3) = 3
(4×1, 3) = 3
(4×2, 3) = 3
9
(9, 7) = 7

(5+7) 12, 19 = 12



```
2
8 8
2 3 7 10 12 15 30 34
1 5 7 8 10 15 16 19
5 4
2 4 5 8 10
4 6 8 9
```

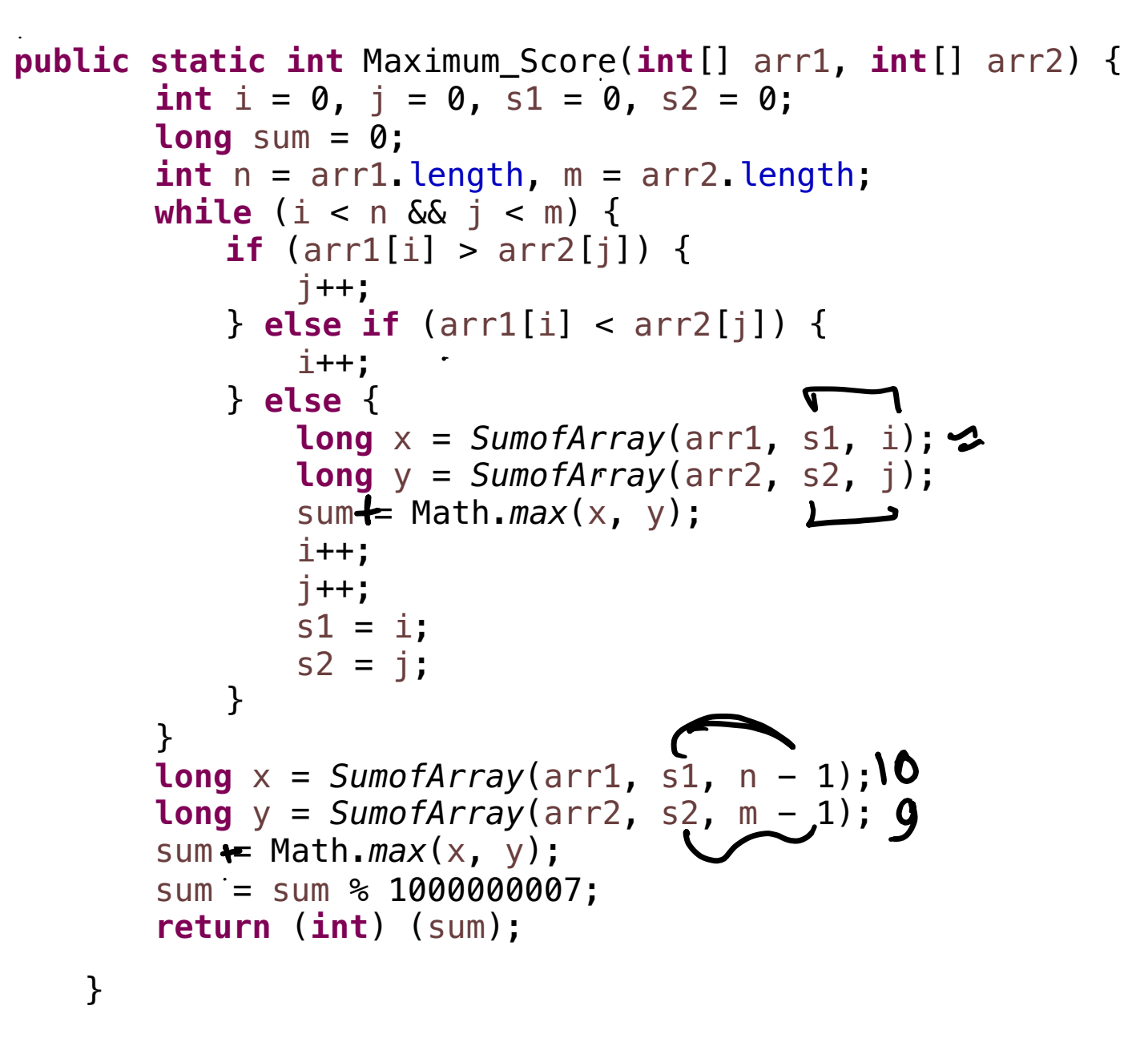
122
2 3 7 10 12 15 30 34
1 5 7 8 10 15 16 19
3
13 + 18 + 27 + 64 = 122

i=0, s1=0
j=0 s2=0

2 3 7 10 12 15 30 34
1 5 7 8 10 15 16 19

s1 - n-1 → 64
s2 - m-1 → 35

int Sum = 13 + 18 + 27 + 64
while(i < n && j < m){
if(arr1[i] > arr2[j]){
j++
}
else if(arr1[i] < arr2[j]){
i++
}
else{
int x = SumofArray(arr1, s1, i)
int y = SumofArray(arr2, s2, j)
Sum = max(x, y) =
i++, j++
s1 = i, s2 = j
}
}



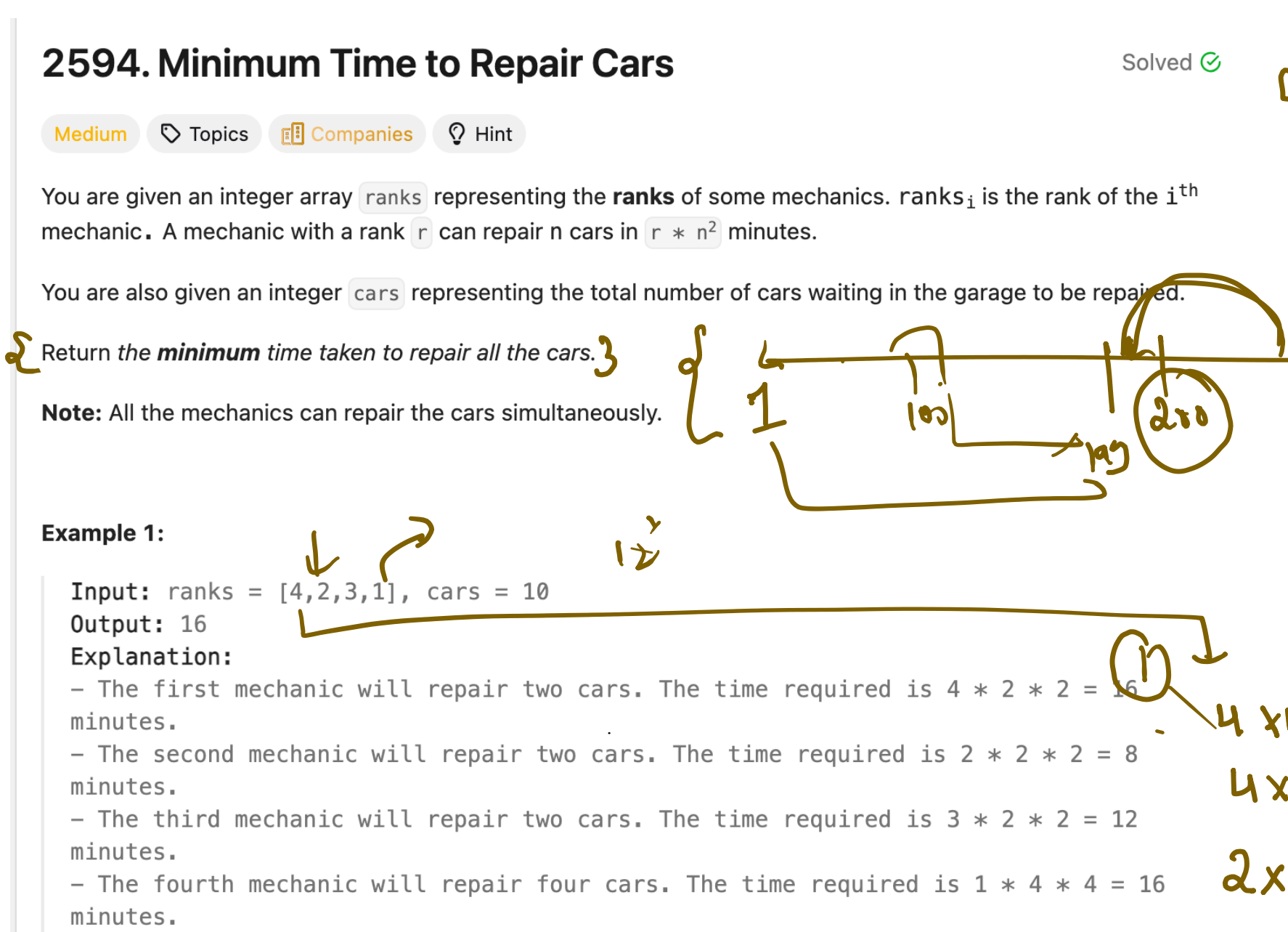
```
public static int Maximum_Score(int[] arr1, int[] arr2) {
        int i = 0, j = 0, s1 = 0, s2 = 0;
        long sum = 0;
        int n = arr1.length, m = arr2.length;
        while (i < n && j < m) {
            if (arr1[i] > arr2[j]) {
                j++;
            } else if (arr1[i] < arr2[j]) {
                i++;
            } else {
                long x = SumofArray(arr1, s1, i);
                long y = SumofArray(arr2, s2, j);
                sum += Math.max(x, y);
                i++;
                j++;
                s1 = i;
                s2 = j;
            }
        }
        long x = SumofArray(arr1, s1, n - 1);
        long y = SumofArray(arr2, s2, m - 1);
        sum += Math.max(x, y);
        sum = sum % 1000000007;
        return (int) (sum);

    }
```

10
9

[2, 4, 5, 8, 10]
s1
[4, 6, 8, 9]
s2

Sum = 6+4+10
= 20



2594. Minimum Time to Repair Cars

Solved ✓

Medium Topics Companies Hint

You are given an integer array `ranks` representing the **ranks** of some mechanics. $ranks_i$ is the rank of the i^{th} mechanic. A mechanic with a rank `r` can repair n cars in `r * n²` minutes.

You are also given an integer `cars` representing the total number of cars waiting in the garage to be repaired.

Return *the* ***minimum*** *time taken to repair all the cars.*

Note: All the mechanics can repair the cars simultaneously.

Example 1:

```
Input: ranks = [4,2,3,1], cars = 10
Output: 16
Explanation:
- The first mechanic will repair two cars. The time required is 4 * 2 * 2 = 16 minutes.
- The second mechanic will repair two cars. The time required is 2 * 2 * 2 = 8 minutes.
- The third mechanic will repair two cars. The time required is 3 * 2 * 2 = 12 minutes.
- The fourth mechanic will repair four cars. The time required is 1 * 4 * 4 = 16 minutes.
It can be proved that the cars cannot be repaired in less than 16 minutes.
```

[4, 3, 2, 1]
$4 \times 10^2 = 400$
1 100 200 400
199
12 min

$4 \times 1^2 = 4$
$4 \times 2^2 = 16$
$2 \times 1^2 = 2$
$2 \times 2^2 = 8$
$2 \times 3^2 = x$
2
2 ← $3 \times 1^2 = 1$
$3 \times 2^2 = 12$
$1 \times 1^2 = 1$
$1 \times 2^2 = 4$
$1 \times 3^2 = 8$

mid = r × (n²)
$\sqrt{\frac{Mid}{r}}$

[4 3 2 1]
0 1 2 3

12 min
12

$4 \times 3^2 = 36$
$4 \times 2^2 = 16$
$4 \times 1 = 4$
1

$3 \times 3^2 = 27$
$3 \times 2^2 = 12$
2

$2 \times 3^2 = 18$
$2 \times 2^2 = 8$
2

$1 \times 3^2 = 9$
$1 \times 4^2 = 16$
3

8

$\sqrt{\frac{12}{4}} = \sqrt{3} = 1$
$\sqrt{\frac{12}{3}} = \sqrt{4}$ 2
$\sqrt{\frac{12}{2}} = \sqrt{6} = 2$
$\sqrt{\frac{12}{1}}$ 3